

Reviewer Pack — Moment-Matrix Spectral Norm Inverse Proportional to Communic...

Entry 9a3f01389b5d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 23:07:10 UTC

Moment-Matrix Spectral Norm Inverse Proportional to Communication Complexity Lower Bound for Max-CUT

Entry ID: 9a3f01389b5d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 23:07:10 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry (Moment Matrices, Positivstellensatz)

Field B (complexity object): Communication Complexity of Max-CUT Instances

Statement:

For a Max-CUT instance on n variables, the spectral norm of its degree- d moment matrix is inversely proportional to the communication complexity lower bound, i.e., $\|M_d\| \geq c \cdot (\log(1/\epsilon)) / (\log n)$, where ϵ is the approximation ratio and c is a constant.

Rationale (proposer’s reasoning):

The spectral norm of the moment matrix reflects the structure of the polynomial constraints in the SOS hierarchy. By linking it to communication complexity, we can uncover how the algebraic structure of the problem influences its communication requirements, potentially revealing new lower bound techniques.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b6a3132f0c84b3f9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + A[i:].index(max(abs(row[i]) for row in A[i:]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def spectral_norm(M):
    n = len(M)
    v = [Fraction(1, math.sqrt(n)) for _ in range(n)]
    for _ in range(100): # Power iteration
        v = [sum(M[i][j] * v[j] for j in range(n)) for i in range(n)]
        v = [x / sum(v[j]**2 for j in range(n))**0.5 for x in v]
    return max(abs(x) for x in v)

def communication_complexity_lower_bound(n, epsilon):
    return Fraction(math.log(1 / epsilon), math.log(n))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 30
    d = 2
    c = Fraction(1, 10) # Hypothetical constant for demonstration

    A = [[random.randint(-1, 1) for _ in range(n)] for _ in range(n)]
    M = [[sum(A[i][j] * A[k][l] for i in range(n) for j in range(n)) for k in range(n)] for l in range(n)]

    M_d = [[M[i][j] ** d if i == j else 0 for j in range(n)] for i in range(n)]
    norm = spectral_norm(M_d)
    lower_bound = communication_complexity_lower_bound(n, Fraction(1, 2))
```

```

ratio = norm / lower_bound
conjecture_holds = ratio >= c

return {
    "metric_name": "Spectral Norm Ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Ratio {ratio} < {c}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std = math.sqrt(sum((r['metric_value'] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd896e7d.py", line 70, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd896e7d.py", line 50, in run_trial
    norm = spectral_norm(M_d)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd896e7d.py", line 31, in spectral_norm
    v = [Fraction(1, math.sqrt(n)) for _ in range(n)]
         ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid Fraction arguments; no data collected to assess conjecture |
next: Fix the Fraction initialization error by using integer denominators

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	64742
2	propose	ollama_remote	qwen3:8b	0	0	111826
3	propose	ollama_remote	qwen3:8b	0	0	84062
4	propose	ollama_remote	qwen3:8b	0	0	91390
5	preregistration	ollama_remote	qwen3:8b	0	0	24092
6	novelty	ollama_remote	qwen3:8b	0	0	20642
7	novelty	ollama_remote	qwen3:8b	0	0	15640
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11246
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10899
10	judge	ollama_remote	qwen3:8b	0	0	20112

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 454652 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9a3f01389b5d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9a3f01389b5d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9a3f01389b5d.tar.gz (if generated)